

GAIADA Social Media

A platform-nest module vertical and department console for organic social publishing — calendar, composer, engagement inbox, and analytics — running on an AGPL-contained publishing engine at arm's length, with mandatory human approval on every action that reaches the public internet. Engineering blueprint and ticket-ready rollout plan.

DOCUMENT Engineering Blueprint v1.0	CODENAME SMM	DATE 2026-07-23	OWNER Social Media
STATUS ● DRAFT FOR REVIEW	CLASSIFICATION Internal		

Contents

00	Executive Summary	08	Console & Capability Matrix
01	Scope & Pillars	09	ERP Integration
02	System Overview	10	Automation Flows
03	Trust Zones & AGPL Containmentment	11	Security
04	Domain Model & Schema	12	Rollout & Tickets
05	Publisher Boundary & Cost Ledger	13	Open Questions
06	Fork/Adapt Plan	14	Decision Log
07	AI Design	15	Glossary

Sections are numbered; each begins on a new page. Use the PDF bookmarks or the page footer to navigate.

§00 Executive Summary

What we are building, why the AGPL boundary is the whole architecture, and the shape of the decision already made.

Social-media becomes a **platform-nest module vertical** (`ModuleContract` key `social` , tables `social_*`) plus a **department console** on the dept-interface-template — the already-reserved "**Publish**" craft group (Calendar · Composer · Inbox · Analytics). The professional moat is **approved platform-app access + brand-fit judgment**, not scheduling plumbing, so the architecture keeps every byte of judgment, tenancy, and approval logic on our side of a hard license line.

The shape, in one paragraph

Postiz is the publishing engine, run AGPL-CONTAINED — an isolated container reached only over its REST API (mere aggregation; our NestJS/Go/Next.js stay uninfected). All domain state, tenancy, RBAC, approvals, and judgment live outside Postiz in platform-nest. Postiz holds exactly two things: network OAuth tokens (it must, to publish and refresh) and the approved-and-scheduled execution queue — **drafts never enter Postiz**. There is **no universal post object**: a `social_posts` master carries the idea/brief/approval state, and `social_post_variants` carry per-network content validated against each network's media rules and quota before anything is queued. A first-class **connector registry** (`social_accounts` + the platform-app fleet) is surfaced exactly like the IT device registry. AI is **local-first via ai-gateway-go** — Hermes for bulk work, Claude only for client-facing polish, brand-voice RAG riding WS8 knowledge, images from the Creative Image Studio, credits-gated. **Human-in-the-loop is mandatory and stricter than SEO**: every outbound public action — publish, reply, delete — is a WS4 `impact: 'high'` suspension, and execution consumes a one-shot `approvalId` with payload-hash match. There is no auto-publish path, for humans or agents. Money — X per-post fees and generative-image/video credits — flows through one `social_usage_ledger` with the SEO-proven stop-loss chain, fail-closed, checked before dispatch.

Build order is risk-honest: **P0 contracts + containment spike** → **P1 organic publish/calendar/composer on our own accounts (\$0, own-risk)** → **P2 engagement inbox** → **P3 AI copy/assets + analytics/reports (+ X metering)** → **P4 agent-proposed drafts**. 27 tickets P0–P4 plus 2 decision-gated, /army-ready, in §12.

DECISION STATE

All architecture-level decisions in this document are **locked** (see §14 Decision Log) and conform to prior foundation-research locks (publisher choice, Chatwoot drop, human-in-the-loop). What remains open are the six owner-decision items in §13. Registration in `MODULES.md` as `social-media · 0.0.0 · PLANNED` happens on approval of this doc.

§01 Scope & Pillars

Three service lines ship in v1; three are explicitly out with a documented seam.

Service lines

SERVICE LINE	V1 DELIVERS	DEFERRED
Organic publishing	Multi-network content calendar, composer with per-network variants + media-rule/quota validation, approval-gated scheduling/publishing via contained Postiz, publish status + failure surfacing, native-post import for calendar completeness	Bulk CSV calendar import; onboarding beyond the core five networks; auto-repost/evergreen recycling
Content creation (copy + assets)	Brand-voice profiles + WS8 RAG corpus per client; AI caption/hashtag/idea drafting (Hermes-first, Claude polish); Creative Image Studio seam for on-brand images (credits-gated)	Generative video (no gateway capability yet); long-form→shorts repurposing (decision-gated SMM-29); carousel auto-design
Community engagement	Unified inbox of comments/mentions (+ DMs where Postiz supports them), AI sentiment/category triage + spike alerts, approval-gated replies, assignment + SLA timers	Reviews platforms; social CRM/audience profiles; auto-moderation rules
KPIs & reporting (cross-cutting)	Outcome-tracked engagements + KPI targets, daily account + per-post metrics via Postiz analytics, monthly client reports (AI narrative → human approve → Drive + deliverable)	Attribution to revenue; competitor benchmarking
Paid social ads	OUT of v1 (locked)	Whole line — seam: <code>social_campaigns.kind</code> reserves <code>'paid'</code>
Social listening	OUT of v1 (locked)	Whole line — seam: inbox sentiment/spike events stand in; a future provider plugs in as a driver
Influencer / UGC	OUT of v1 (locked)	Whole line — seam: creators as agency-vertical contacts + briefs

Non-goals (v1)

- **No adopted external UIs** — Postiz's web UI is never exposed; the SMM console is the only operator surface.
- **No direct vendor AI calls** — everything through ai-gateway-go (locked).
- **No second inbox stack** — Chatwoot dropped; wa-chat-bot stays as-is for WhatsApp/Telegram, untouched.
- **No auto-publish, ever** — not a config option, not for agents, not for "trusted" flows. The WS4 gate is structural, not policy.
- **No paid-ad API integration, no listening data purchase, no influencer tooling.**
- **No client-facing portal changes** — reports deliver as files (Shared Drive + `deliverables`).

Architectural pillars

P1 · POSTIZ CONTAINED

The publishing engine runs at arm's length behind one REST adapter. No shared code, no shared DB, no linking — the fork stays thin-to-zero.

P2 · TWO-LEVEL CONTENT

No universal post object. A master idea/brief/approval row; per-network variants carry content, media, and settings, validated before anything is queued.

P3 · CONNECTOR REGISTRY

Per client × network — app-review status, token health, quota counters, last error — surfaced exactly like the IT device registry.

P4 · LOCAL-FIRST AI

Hermes for bulk drafting, Claude for client-facing polish only, brand-voice RAG over WS8 knowledge, images via the Creative Image Studio.

P5 · HUMAN-IN-THE-LOOP, ALWAYS

Every outbound public action is a WS4 suspension; execution consumes a one-shot approval with payload-hash match. No exceptions for agents.

P6 · ONE METERED LEDGER

X per-post fees and generative credits flow through a single ledger with fail-closed stop-loss — engagement budget → tenant cap → global cap.

FIT WITH PRIOR DECISIONS

Conforms to the ERP holding-OS vision (enablement via `enabled_modules` OR active `service_assignment`, WSD-3 module-sliced RLS third wall). Lands on the dept-interface-template exactly as reserved: SMM → "Publish": Calendar · Composer · Inbox · Analytics — this design builds those four pages and only then registers the toolkit. Reuses `clients`, `deliverables`, `time_entries`, `files`, `projects`, the Creative Image Studio asset tables, the WS4 approvals surface, and the SEO-proven ledger/stop-loss + one-shot-approval patterns.

§02 System Overview

The whole subsystem on one plate: the private platform, the license zone at arm's length, and every seam between them.

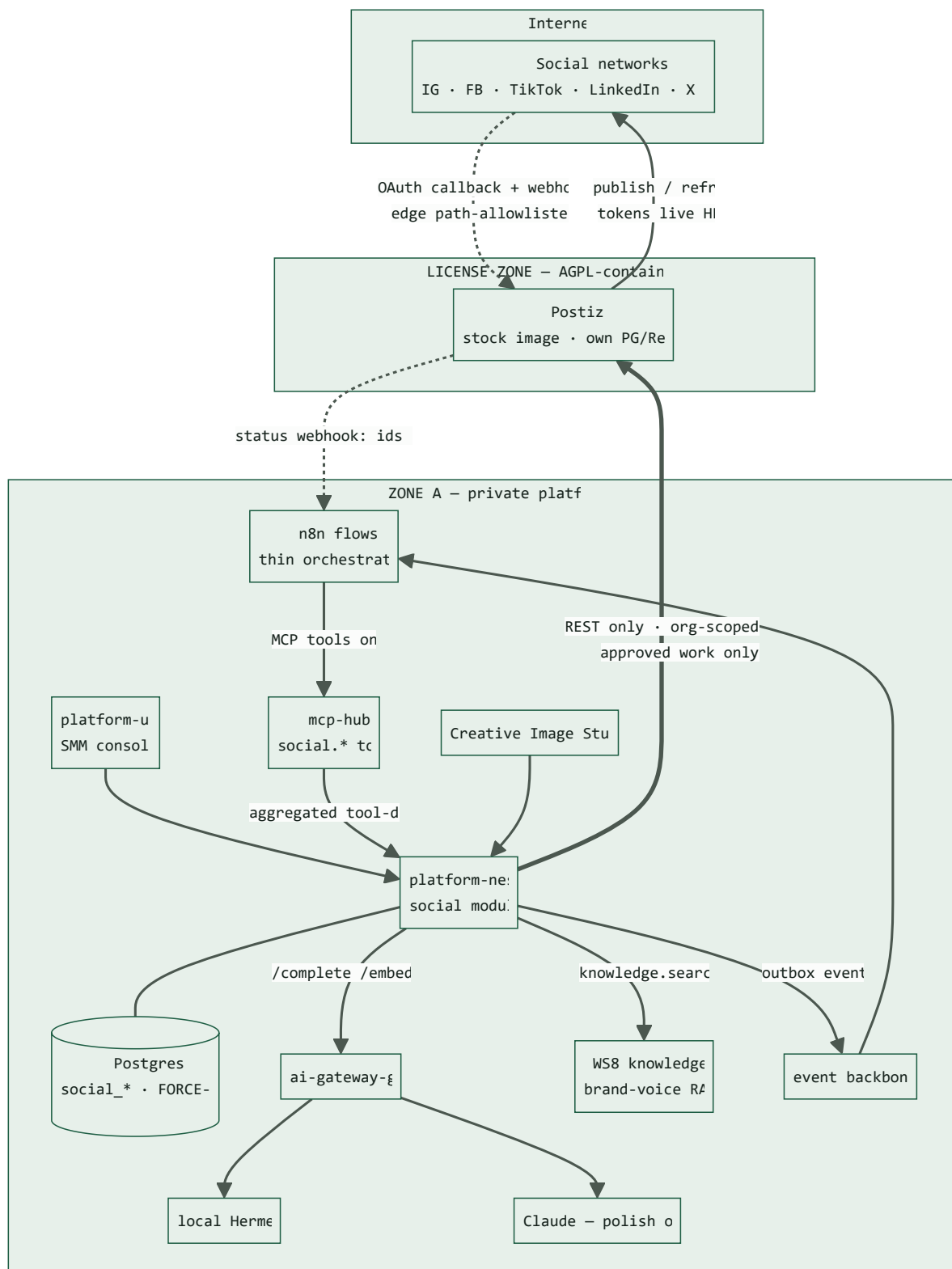


FIG. 02-1 – Master system diagram. Bold seam = control (REST, org-scoped, approved work only). Dotted seam = events/callbacks (validated, notification-only). The AGPL boundary is a **license zone**, not a network zone – Postiz runs on the private network but is architecturally at arm's length.

Reading the diagram

- **Zone A** holds all state and judgment: the module, its RLS-scoped Postgres, the AI routing, the brand-voice RAG, the MCP tool surface, and the event backbone.
- **The license zone** is Postiz — reached exclusively over REST, no shared code, no shared schema, no in-process embedding. Its own Postgres and Redis are opaque to us; we never query them.
- Two arrows cross to the internet: Postiz's publish/refresh egress to the networks (where the network tokens live), and a narrow path-allowlisted ingress (edge proxy → Postiz's OAuth-callback and platform-webhook paths only).
- Status webhooks toward us are **notification-only** (post/integration ids) — the authoritative state is always re-fetched over the authenticated Postiz API, never trusted from the webhook body.

§03 Trust Zones & AGPL Containment

Follows the WebDesk zone doctrine, extended with the license zone this subsystem introduces.

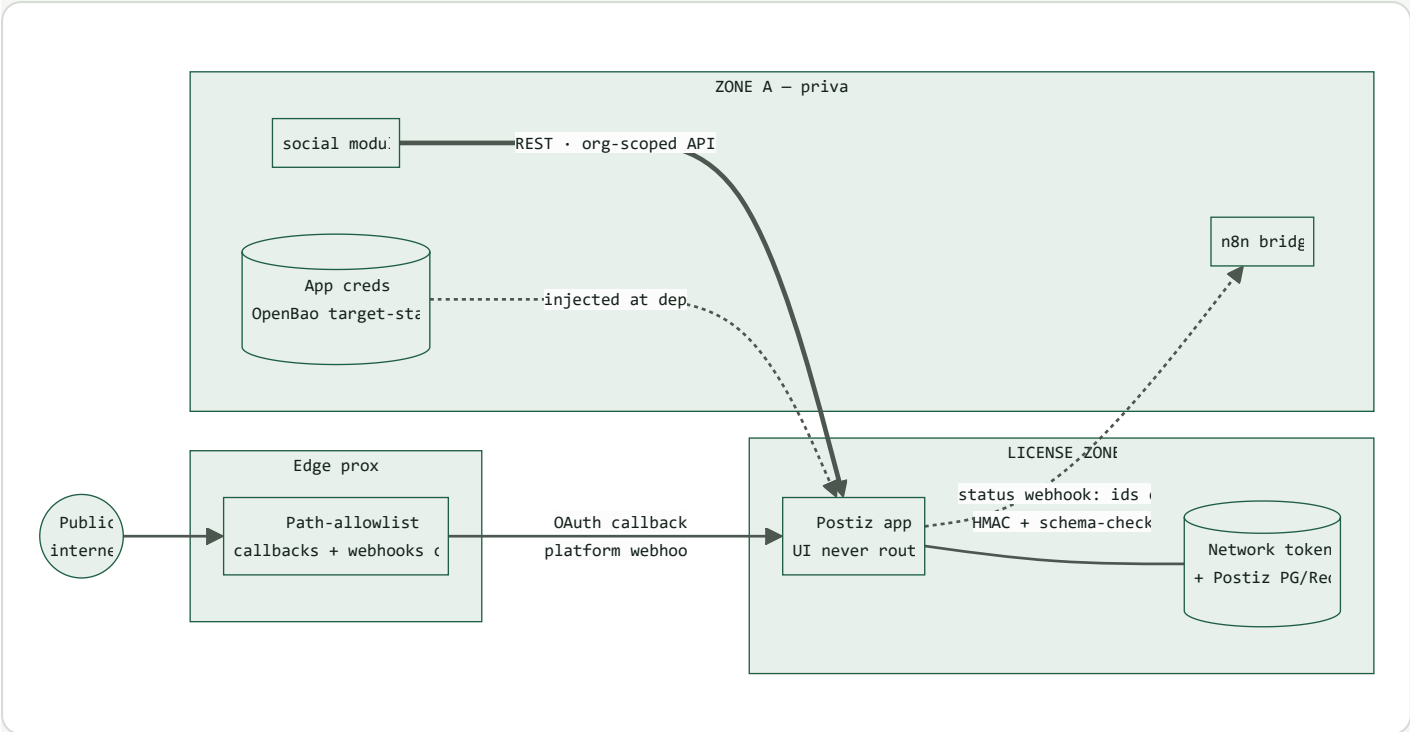


FIG. 03-1 – Network + license boundary. Only Postiz's callback/webhook paths are reachable from outside; its admin UI and general API are never routed. Network tokens live inside Postiz only – never copied into our DB or vault.

Surfaces & rules

SURFACE	ZONE	RULES
social module + ledger + registry	A	Postiz org-scoped keys + our platform-app credentials are server-side env → OpenBao target-state, held by platform-nest / injected into the Postiz container. Never in the browser, never in n8n credentials, never per-tenant rows.

SURFACE	ZONE	RULES
Postiz (publisher engine)	A network L license	Isolated compose services (app + own Postgres + own Redis, per the db-topology isolation doctrine). Reached ONLY via REST from platform-nest. UI not routed anywhere; DB opaque to us. Containment invariants: no shared libraries/schema/embedding; tenancy/RBAC/approvals entirely on our side; fork thin-to-zero; source-offer footer link.
Network OAuth callbacks + platform webhooks	edge → Postiz	Exact-path allowlist (everything else 404s), TLS-terminated, rate-limited. The single inbound door; a compromise reaches Postiz, not platform data — and Postiz holds only social tokens.
Postiz → us status webhooks	edge → n8n	Signed-webhook bridge pattern: HMAC/secret-path, schema-validated, treated as triggers only — n8n calls an MCP tool; the module re-fetches authoritative status from Postiz's API.
Client social accounts (tokens)	inside Postiz only	Created by Postiz's OAuth flows, stored in its encrypted store, refreshed by it, and never copied into our DB or vault. Our registry mirrors state-about, never the secret.
Creative Image Studio / gateway image seam	A	Generative calls ride ai-gateway-go (budget caps, DLP, egress audit apply automatically); outputs land as files rows.
wa-chat-bot	untouched	Explicitly out of scope; WhatsApp/Telegram engagement stays there — locked, no social traffic routed into it.

CONTAINMENT INVARIANT

There is no network route and no credential by which a compromise of Postiz — or a Postiz CVE — can reach company data or the module's judgment. Postiz receives only approved publishable content and returns only public-surface data (comments, metrics). The license boundary and the security boundary coincide by construction.

§04 Domain Model & Schema

The core entities, the two-level content model, and the FORCE-RLS third wall every table inherits.

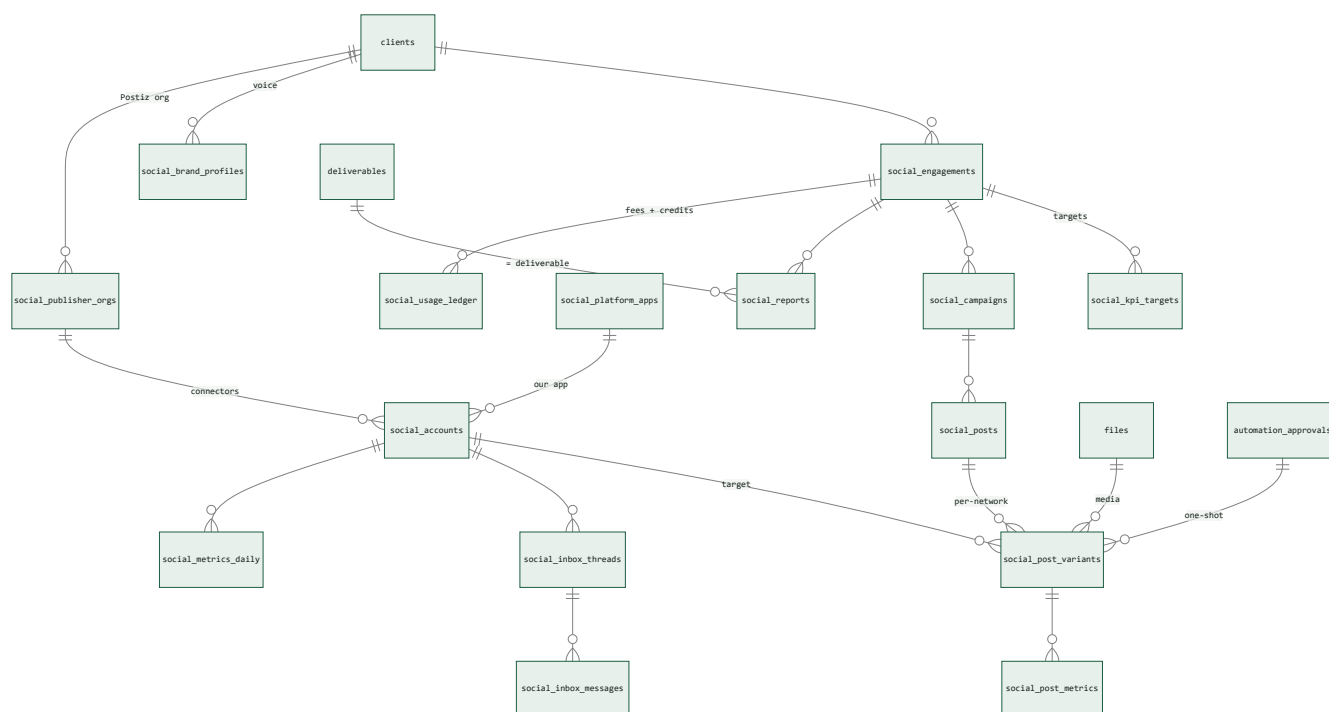


FIG. 04-1 - Entity map. Every arrow = one-to-many. `social_posts` / `social_post_variants` is the locked two-level content model - there is no universal post object.

Design rules (inherited, not optional)

- Every tenant table: `tenant_id uuid NOT NULL REFERENCES companies(id), origin_site, created_at/updated_at`, soft-delete `deleted_at` where user-facing.
- **FORCE-RLS with the WSD-3 third wall** — policy predicate `tenant_id = ANY(app_current_tenants()) AND app_module_allowed('social')` on both USING and WITH CHECK, byte-identical to `0028_module_hr.sql`. App side reaches these tables only via `withTenants(tenants, { modules: ['social'] })`.
- Migrations take the next unused number (0034+ as of 2026-07-23 — coordinate with any in-flight SEO tickets drawing from the same sequence); no in-migration GRANTS; no `sync_app` grants (social tables do not sync in v1).
- Money: metered cost is `numeric(12,6)` USD (same rationale as the SEO ledger). No client-facing money in v1 — no ad spend.
- The single non-tenant table is `social_platform_apps` — our own app fleet, no client data, admin-endpoint-only. Unlike SEO there is **no shared market-data cache**: every other byte in this module is client-private.

Tables

ENTITY	CARRIES	SCOPE
social_engagements	Outcome-tracked client engagement · tool_scope jsonb (networks/posting/inbox/ai/reporting toggles) · usage_budget_usd	per client
social_publisher_orgs	One Postiz org per (tenant, client) · postiz_org_id · api_key_ref · UNIQUE both ways	per client

ENTITY	CARRIES	SCOPE
social_accounts	Connector registry: network · handle · status · quota jsonb · capabilities jsonb · health	per account
social_platform_apps	Our approved developer-app fleet: review_status, scopes, quota_regime, credential_ref	global, no RLS
social_brand_profiles	Voice/tone jsonb, hashtag_strategy, knowledge_source_ids → WS8 corpus pointers	per client
social_campaigns	Content campaigns/themes · kind ('organic' now, 'paid' reserved)	per engagement
social_posts	Master idea/brief/approval-state row · source (human/ai/agent/native_import) · rolls up variant states	per engagement
social_post_variants	Per-network body/media/settings · validation jsonb · payload_hash · approval_id · postiz_post_id	per post × account
social_inbox_threads / _messages	Comments/DMs/mentions · sentiment (AI-stamped) · SLA · assignment · reply drafts (one-shot gated)	per account
social_kpi_targets	Per-engagement metric targets: followers, reach, engagement rate, response time, etc.	per engagement
social_metrics_daily / _post_metrics	Account time-series / per-post performance snapshots (append-only)	per account / variant
social_reports	Monthly/campaign/adhoc report — metrics snapshot + AI narrative → human-edited → deliverable	per engagement
social_usage_ledger	X fees + AI credits, metered, stop-loss chain (\$05)	per engagement

```

-- social_post_variants - state law (normative)
-- payload_hash = hash(body, first_comment, media, settings, scheduled_at)
--
-- queued    ⇐ ONLY by consuming an approved one-shot approval_id
--             whose hash matches this row's current payload_hash
-- any edit ⇒ approval_id := NULL, status := 'draft', payload_hash recomputed
-- native_import posts enter directly as 'published' with postiz_post_id NULL
--             (calendar completeness, not execution - no approval trail faked)

```

NO SHARED CACHE – DELIBERATE CONTRAST WITH SEO

The SEO module's no-RLS `search_data_cache` exists because market data is public-world and reusable cross-tenant. Nothing in SMM qualifies: posts, inbox threads, metrics, and account state are all client-private. Analytics pulls are cheap (\$0 via Postiz), so caching lives inside tenant rows — `social_metrics_daily` is the cache. D-4: no RLS exemptions in this module beyond the client-data-free `social_platform_apps` admin table.

§05 Publisher Boundary & Cost Ledger

One adapter is the entire AGPL containment line; one ledger is the entire money-safety line.

PURPOSE	The single, capability-based interface every publishing operation goes through. Keeping every call at arm's length behind one adapter is the AGPL containment line.
TECHNOLOGY	platform-nest, <code>src/modules/social/publisher/</code> . One driver ships in v1 (Postiz); the port exists so the Mixpost-Pro fallback (or a direct-API driver) is a driver swap, not a redesign.
RESPONSIBILITIES	- Org lifecycle: <code>createOrg</code>, <code>connectUrl</code> (OAuth entry), <code>listIntegrations</code> (registry sync source) — tenant mapping stays ours; the driver carries only the opaque org ref. - Publishing: <code>schedulePost</code>, <code>cancelPost</code>, <code>getPostStatus</code> — accepts only approved work; the caller enforces \$07, the driver asserts <code>approvalId</code> present. - Engagement: <code>listComments</code>, <code>sendReply</code>. - Analytics: <code>getAccountMetrics</code>, <code>getPostMetrics</code>. - Money: <code>estimateCostUsd(op)</code> — consulted before dispatch (X pay-per-post).
INTERFACES	in: module call sites (dispatch choke-point, registry sync, inbox pull) · out: HTTP+JSON to Postiz's public API only
DATA OWNED	None — stateless per call; org-scoped API key resolved from <code>api_key_ref</code> at call time
SECURITY	Every request/response OTel-annotated (network, org, op, cost_usd); no Postiz type or code imported — containment is enforceable by lint (zero <code>postiz</code> package deps)
OPERATIONS	Driver swap (Mixpost fallback) = new driver + deploy change, not a redesign

The usage/cost ledger

X is a metered cost center (~\$0.015/post, ~\$0.20 with link — reverify at build time) and generative image/video is the highest-cost AI class. Both flow through one ledger, mirroring `search_provider_calls : social_usage_ledger` (`kind`

in `x_post / ai_image / ai_video / ai_cloud_text`, `cost_usd numeric(12,6)`, `status`, `requested_by`, `correlation_id` — indexed (`tenant_id`, `engagement_id`, `created_at DESC`).

Scope + budget stop-loss (fail-closed, one choke-point before dispatch)

Byte-for-byte the SEO pattern:

- 1. The capability must be enabled in the engagement's `tool_scope` (`networks.x`, `ai.imageGen`) — a disabled toggle refuses **with the toggle named**, regardless of budget.
- 2. Month-to-date `SUM(cost_usd)` vs — engagement `usage_budget_usd` → tenant monthly cap → global platform cap (env, default \$100/mo until X usage is proven).
- 3. Breach: refuse + emit `social.usage.budget_threshold` (also at 80%); console shows the blocked state; override = `social:credits:admin` + audit.

PRICE OF THE CLICK

The composer surfaces `estimateCostUsd` on every X variant **before** approval is requested, so the approver sees the price of the click. Ledger powers the per-engagement usage panel, tenant rollup, and the `social.usage_cost.month` rollup metric. Gateway-side AI cost caps remain enforced upstream independently (defence in depth).

§06 Fork/Adapt Plan Per Repo

"Contained" = we run it at arm's length as an execution engine; "data-only" = we take reference/technique, never run it.

REPO	VERDICT	KEEP	STRIP / NEVER RUN	ADD / HOW IT PLUGS IN
gitroomhq/postiz-app (AGPL-3.0)	Adopt · AGPL-CONTAINED (load-bearing). Stock upstream image, isolated compose stack, tenancy mapped in <i>our</i> <code>social_publisher_orgs</code>	14-network connectors, OAuth + token refresh, media upload, queue/retry, comments, analytics endpoints	Its web UI (never routed); its user/org self-signup (disabled); any temptation to edit its Prisma schema	Compose services on the private network; edge exposes only callback/webhook paths; app creds injected via OpenBao; <code>SocialPublisher</code> driver; source-offer footer link

REPO	VERDICT	KEEP	STRIP / NEVER RUN	ADD / HOW IT PLUGS IN
— fork-touchpoint budget (thin-fork line)	Allowed ONLY: build/compose/env packaging; disabling signup/UI via config; outbound status-webhook emitter if missing upstream; security backports	—	—	Tripwires → Mixpost fallback: touching its DB schema/tenancy; embedding our authz inside it; patch surface > ~5 files or unrebasable in a day; a licensing-counsel red flag
inovector/mixpost (Pro, proprietary)	FALLBACK only — do not run in v1	11-network coverage + built-in inbox as the driver-port checklist	Everything (not deployed)	A mixpost driver behind the same SocialPublisher port
chatwoot/chatwoot	DROPPED (locked) — overlaps wa-chat-bot	Its triage/assignment UX informs the Inbox tab	The app	—
ClipsAI/clipsai (MIT)	Decision-gated (SMM-29, not mobilized)	Python lib as job-mode worker; reuses local faster-whisper	Any service mode	Job container → clips land as files
SamurAIGPT/AI-Youtube-Shorts-Generator (MIT)	Reference-only	Highlight-selection prompt patterns	—	Folded into SMM-29 design when gated open
Creative Image Studio (internal)	Reuse + extend at the seam	Grading engine, presets, asset library	—	"Send to SMM composer" flow + gateway generative-image seam behind credits + WS4

AGPL containment — the five invariants (normative)

1. Isolated service/container, REST-only interaction.
2. Multi-tenancy + all proprietary logic outside the Postiz process.
3. Thin-to-zero fork within the touchpoint budget.
4. Source offer to interacting users via console footer link.
5. Postiz frontend never served to anyone.

LEGAL SIGN-OFF

Legal sign-off on this containment memo is OQ-3 (§13). The Mixpost fallback price of failure is known and bounded — if any tripwire fires, the swap is a driver + deploy change, not a redesign.

§07 AI Design

Task-to-model routing, the brand-voice RAG, and the WS4 spine — stricter than SEO because a bad post ships to the world.

Task → model routing (all via ai-gateway-go; never direct vendor calls)

TASK	MODEL	TRIGGER	NOTES
Caption/copy drafting	Hermes → Claude	Composer	Claude only when <code>tool_scope.ai.cloudPolish</code> ; brand profile + WS8 brand-voice RAG
Hashtag generation	Hermes	Composer	Constrained by <code>hashtag_strategy</code> ; per-network counts/placement
Content ideas / angles	/embed + Hermes	Calendar planning	Clusters own top-performing posts; no external trend data (listening is out)
Comment/DM triage	Hermes + rules	<code>smm-inbox-pull</code>	Stamps sentiment; urgent/negative spike → <code>social.sentiment.spike</code>
Reply drafting	Hermes	Inbox	Draft rows only; brand-voice RAG applied; sending always WS4-gated
Best-time-to-post	Classical stats, no LLM	Weekly job	Regression/heatmap over own metrics; suggestion chip, never auto-applied
Image generation	Studio + gateway seam	Composer/Studio	Highest-cost class — <code>tool_scope.ai.imageGen</code> + credits ledger; asset still rides the post's WS4 approval
Brand-safety pre-check	vision + Hermes	Pre-submit	Advisory flag only (<code>validation.warnings[]</code>), never a silent block
Report narrative	Hermes → Claude	Monthly flow	The one place cloud polish defaults on (client-facing)
Video repurposing	faster-whisper + Hermes + ClipsAI	On demand	Decision-gated (SMM-29) — not in v1 scope

BRAND-VOICE RAG (D9 HYGIENE)

The corpus (approved past posts, brand guidelines, tone docs) is ingested as tenant-ACL'd WS8 knowledge sources — WS8 stays the sole owner of derived knowledge stores;

`social_brand_profiles` stores only `knowledge_source_ids` pointers. Published posts feed back into the corpus on delivery, continuously sharpening drafts. No new pgvector columns in this module.

AI-drafts → human-approves → publish (the WS4 spine)

1. AI (or a human, or a WS8 agent) produces **draft rows** — always persisted, never dispatched. Drafts live in our DB only; nothing enters Postiz at draft time.
2. Low-impact artifacts (reports, campaign plans) approve in-console via module permissions. **Anything outbound-public** — publish, reply, cancel-published, delete — goes through WS4: the MCP tool is declared `write:true, impact:'high'`, the hub write-gate suspends it into `automation_approvals`, with a full preview (rendered variant, media, target account, scheduled time, estimated X cost).
3. **Execute — one-shot, single-mode:** approval produces an `approvalId` bound to the variant's `payload_hash`. The dispatch choke-point consumes it (status approved, hash match, unconsumed) in the same transaction that calls `schedulePost` and stamps `postiz_post_id` — exactly once, replay-refused. Any edit after approval nulls the approval and the variant re-enters review.
4. **Scheduling semantics:** approval covers content + target + time window. Postiz executes the queue; a reconcile flow re-fetches authoritative status. A variant stuck queued past its slot + grace raises `social.post.failed` for human action — never auto-retry a publish that may have half-succeeded.

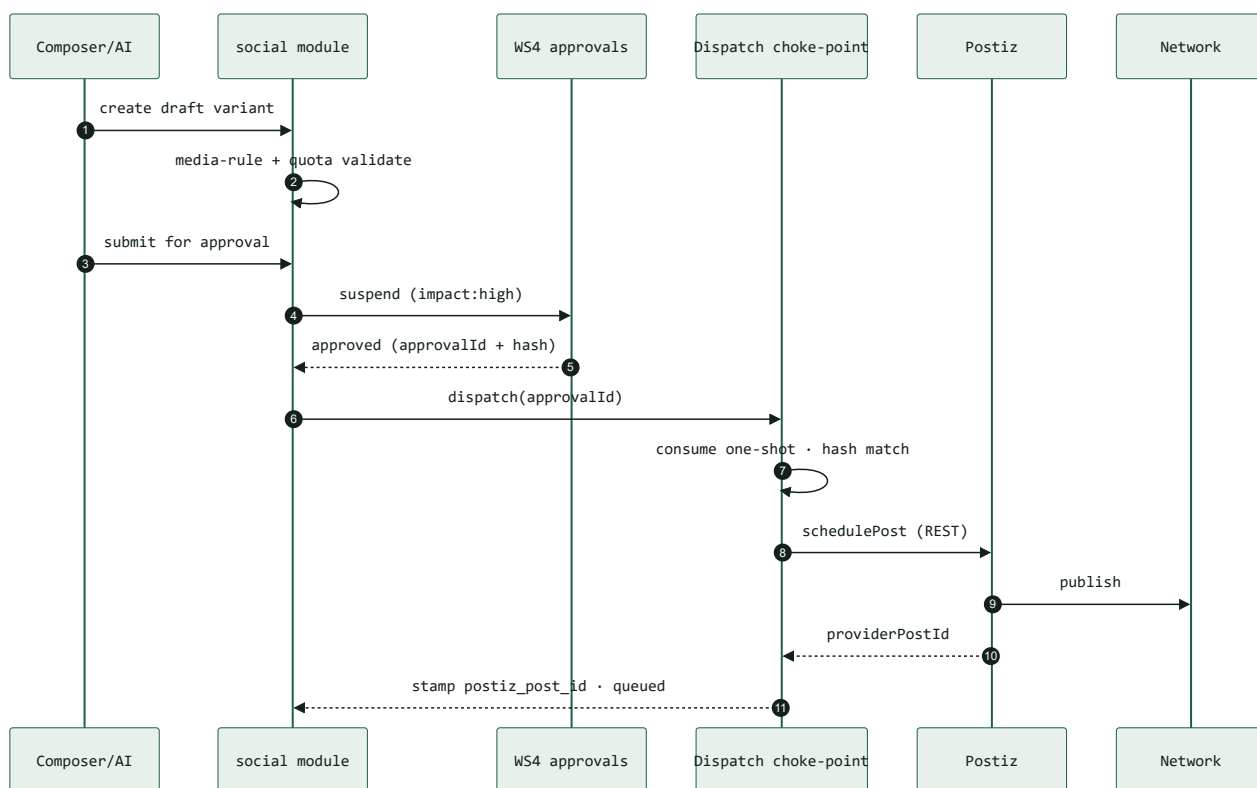


FIG. 07-1 – Draft → WS4 approval → one-shot dispatch. No approval, no publish – including for humans in the console. Any edit after approval nulls the approval and the hash is recomputed.

Social data + actions as MCP tools (WS8 agents)

Registered via `ModuleContract.mcpTools` (aggregated by mcp-hub — nothing hub-side to hardcoded). Reads are `minAssurance:'low'`; credit-spending drafts are `write:true, impact:'medium'`; everything outbound-public is `impact:'high'`:

TOOL	KIND	IMPACT
<code>social.listAccounts / queueSummary / inboxSummary / metricsSummary / ledgerSummary</code>	read	—
<code>social.draftPost / draftReplies / generateHashtags / suggestPostTime / draftReport</code>	AI draft	low (writes drafts only)
<code>social.generateImage</code>	credit spend	medium (tool_scope + ledger stop-loss)
<code>social.pullInbox / pullMetrics / syncAccountHealth</code>	\$0 sync trigger	low
<code>social.submitForApproval</code>	workflow	low (creates the suspension)

TOOL	KIND	IMPACT
<code>social.publishPost</code> / <code>sendReply</code> / <code>cancelScheduled</code> / <code>deletePublished</code>	outbound public	high → always suspends

This is the P4 agent surface: a WS8 agent can research, draft a week of posts, generate imagery (within credits), and file everything into the approval queue — and can never publish.

§08 Console & Capability Matrix

One department console on the dept-interface-template, and the button-level matrix that decides what every action needs.

Department name **"SMM"** (already seeded in the agency org structure), slug `smm`, on the two-level dept-interface-template exactly as Web Dev/Creatives use it: universal spine **Home · Work · Connections** + the already-reserved craft group **"Publish"** — this design changes nothing about that IA, it builds it.

GROUP	SUB-TABS (ROUTE UNDER <code>/DEPARTMENTS/SMM/</code>)
Publish	Calendar (<code>calendar</code> , full-bleed) · Composer (<code>composer</code>) · Inbox (<code>inbox</code>) · Analytics (<code>analytics</code>)

Per the toolkit file's own rule, the SMM toolkit is added to `TOOLKITS` only when all four routes exist. Home = the command-center template (KPI strip: posts scheduled this week, awaiting approval, inbox open + SLA-at-risk, connected accounts + health, MTD metered spend). Connections gains the account-connect flow + connector-health states.

Tab intents

CALENDAR Month/week grid of master posts + variant chips, drag-reschedule (re-approval on hash change), per-network quota strips, campaign swimlanes.	COMPOSER Master post + per-network variant editor, media-rule validation inline, brand-voice AI drafting, Studio attach, submit-for-approval.	INBOX Triage queue (sentiment/urgency sort), thread view, assignment + SLA timers, reply drafts + approval states.
ANALYTICS Account trend charts, per-post leaderboard, KPI-vs-target, usage/ledger panel.		

Button capability matrix

Legend — what each action needs: **AI/OWN-STACK** local Hermes + gateway + our data, usable immediately, \$0 external · **ACCOUNT LINK** client network account connected via contained Postiz and our platform app approved for that network · **METERED** X per-post fee or generative credits, ledger stop-loss applies · **WS4 APPROVAL** human decision required, one-shot execution. Every action is additionally gated by the engagement's **tool_scope**: a disabled network/feature renders the button disabled and names the missing toggle.

CONSOLE ACTION	TAB	NEEDS	GATE
Create engagement / KPI targets / campaign	Home/Calendar	—	permission only
Configure engagement scope & budget	Home	—	social:scope:write
Connect a client social account (OAuth via Postiz)	Connections	ACCOUNT LINK	social:account:connect
View connector registry / health / quota	Connections	—	read permission
Create post + per-network variants	Composer	—	social:post:write
Validate media rules / quota pre-check	Composer	AI/OWN-STACK	automatic on edit
AI draft captions / hashtags / ideas	Composer	AI/OWN-STACK	draft only
Claude polish (client-facing copy)	Composer	AI/OWN-STACK (gateway-capped)	draft only
Generate image (Creative Image Studio seam)	Composer	METERED	draft only
Attach media from files/Drive/Studio library	Composer	AI/OWN-STACK	—
Best-time suggestion chip	Composer	AI/OWN-STACK	advisory only
Submit post for approval (preview + X cost shown)	Composer	—	creates WS4 suspension

CONSOLE ACTION	TAB	NEEDS	GATE
Approve & schedule / publish now	Approvals inbox	ACCOUNT LINK + METERED if X	one-shot
Reschedule an approved post	Calendar	—	hash change → re-approval
Cancel a scheduled (unpublished) post	Calendar	ACCOUNT LINK	one-shot (outbound-state mutation)
Delete a published post	Calendar/Analytics	ACCOUNT LINK	one-shot (public, irreversible)
Record a natively-posted item (import)	Calendar	—	<code>social:post:write</code> (bookkeeping)
Pull inbox now / auto-sync comments+DMs	Inbox	ACCOUNT LINK	— (\$0 read)
AI triage sentiment / draft replies	Inbox	AI/OWN-STACK	draft only
Send reply (comment/DM)	Inbox	ACCOUNT LINK	one-shot
Assign thread / set status / escalate	Inbox	—	<code>social:inbox:reply</code>
Pull account + post metrics	Analytics	ACCOUNT LINK	— (\$0 read)
Generate report (metrics + AI narrative)	Analytics	AI/OWN-STACK	<code>social:report:approve</code>
Deliver report (Shared Drive + deliverable)	Analytics	AI/OWN-STACK	after approve
View usage ledger / raise engagement budget	Home/Analytics	—	<code>social:ledger:read</code> / <code>social:credits:admin</code>
Manage platform-app fleet	Admin (not dept console)	—	<code>social:apps:admin</code>

P1 DOGFOOD

Everything green plus the composer/calendar loop against our own brand accounts ships value in P1 with zero external spend and zero client risk — deliberate: own accounts are the dogfood + demo surface while client app reviews grind through OQ-1.

§09 ERP Integration

Every existing platform subsystem this module plugs into — nothing new to invent, everything reused.

SUBSYSTEM	INTEGRATION (CONCRETE)
platform-nest	<code>ModuleContract</code> key <code>social</code> ; controller <code>@Controller("api/:tenantId/modules/social")</code> (hr convention); <code>ModuleEnabledGuard</code> ; enablement via <code>enabled_modules</code> OR active <code>service_assignment</code>
BFF contract	New section in <code>FRONTEND-BFF-CONTRACT.md</code> : engagements, scope, brand-profiles, accounts/connect, campaigns, posts/variants/submit, calendar, inbox threads/reply, kpi-targets, metrics, reports, ledger — shapes canonical in <code>platform-ui/src/lib/social.ts</code>
Postiz (contained)	The <code>SocialPublisher</code> Postiz driver is the ONLY caller; compose stack + narrow edge ingress; org-per-client mapping in <code>social_publisher_orgs</code>
mcp-hub	<code>social.*</code> tools via <code>mcpTools</code> aggregation (§07); no hub changes needed
ai-gateway-go	<code>/complete</code> (Hermes default, Claude flagged) + <code>/embed</code> + vision (brand-safety) + generative-image seam; budget caps + DLP + egress audit apply automatically
WS8 knowledge	Brand corpus as tenant-ACL'd knowledge sources; drafting retrieval via <code>knowledge.search</code> (D9 preserved); delivered posts fed back into the corpus
Creative Image Studio	Asset attach flow + generative seam; assets keep their creative-asset lineage; credits metered in the §05 ledger
automation (n8n)	Flows in §10; backbone rule respected — n8n orchestrates, MCP accesses, zero logic in workflows
WS4 approvals	Every outbound-public <code>social.*</code> tool suspends into <code>automation_approvals</code> ; execution consumes a one-shot <code>approvalId</code> + payload-hash match; approval preview renders the exact variant
Event backbone	<code>social.post.published/failed/awaiting_approval</code> , <code>social.account.expiring/error/quota_near</code> , <code>social.inbox.sla_breach</code> , <code>social.sentiment.spike</code> , <code>social.usage.budget_threshold</code> , <code>social.report.ready_for_review/delivered</code> → notifications bell + n8n bridge
Files / Shared Drive (WS11)	Post media + produced creative + rendered reports as <code>files</code> rows mirrored to the client's Drive folder
Rollups (D12)	Metrics: accounts connected, posts published/month, approvals pending, inbox open, followers total, usage cost/month (USD), reports delivered
Cerbos	New resource policies (§11); UI capabilities mirrored in <code>lib/rbac.ts</code> (defence-in-depth, Cerbos authoritative)

SUBSYSTEM	INTEGRATION (CONCRETE)
observability (WS9)	OTel spans on every publisher call (network, org, op, cost_usd), publish latency, connector-health gauge, inbox SLA gauge, ledger-vs-cap gauge; fail-soft <code>OTEL_ENABLED</code>
wa-chat-bot	Explicit non-integration (locked): WhatsApp/Telegram stay there; no social traffic crosses
Future service-line seams	Paid ads → <code>social_campaigns.kind='paid'</code> + the SEO change-proposal template; listening → provider driver + <code>social_mentions</code> ; influencer → agency-vertical briefs. Documented, not built

§10 Automation Flows (n8n / WS4)

Thin orchestrations calling `social.*` MCP tools — publishing itself is not an n8n flow.

All flows live in `automation/workflows/`, kebab-named like the existing set. **Scheduling is scope-driven:** each flow iterates only engagements whose `tool_scope` enables the feature (the module filters; n8n stays logic-free per the backbone rule). Publishing itself dispatches transactionally at approval-consumption inside the module (\$07) — n8n only reconciles and reacts.

FLOW	SCHEDULE / TRIGGER	MCP CALLS	PHASE
<code>smm-post-status-sync</code>	Postiz webhook (ids) + 15-min safety poll	<code>social.syncPostStatus</code> (authoritative re-fetch)	P1
<code>smm-connector-health</code>	daily	<code>social.syncAccountHealth</code> → expiring/error/quota events	P1
<code>smm-inbox-pull</code>	every 15 min per enabled engagement	<code>social.pullInbox</code> (idempotent upsert)	P2
<code>smm-sentiment-triage</code>	event <code>social.inbox.new_batch</code>	<code>social.triageInbox</code> → spike detection → notify/escalate	P2
<code>smm-inbox-sla-guard</code>	every 15 min	<code>social.inboxSummary</code> → SLA breach events → notify	P2
<code>smm-metrics-daily</code>	nightly	<code>social.pullMetrics</code> (accounts + recent posts)	P3
<code>smm-usage-guard</code>	daily	<code>social.ledgerSummary</code> → 80%/100% threshold events	P3
<code>smm-monthly-report</code>	monthly per engagement	<code>social.draftReport</code> → review → <code>social.deliverReport</code>	P3
<code>smm-best-time-refresh</code>	weekly	<code>social.refreshPostTimeModel</code> (classical stats job)	P4

FLOW	SCHEDULE / TRIGGER	MCP CALLS	PHASE
smm-agent-content-brief	weekly per opted-in engagement	WS8 agent: draft posts + imagery within credits → <code>social.submitForApproval</code>	P4

§11 Security

The threat model and control matrix — where the license boundary and the security boundary are the same line.

Threat model

THREAT	VECTOR	PRIMARY CONTROL	SEVERITY
AGPL/license contamination	Fork drift, shared code/DB with Postiz	REST-only adapter, lint-enforced zero Postiz deps, fork-touchpoint budget	Critical
Wrong-account publish	Tenant-mapping bug across the FK chain	Validated at composer AND dispatch; org UNIQUE; fail-closed refusal + audit	Critical
Accidental / malicious public post	Auto-publish path, replay, TOCTOU	WS4 gate structural (not policy); one-shot approvalId + payload-hash match	Critical
Network token theft	Compromise of our DB/vault	Tokens live inside Postiz only, never copied out — disjoint blast radii	High
Forged status webhook	Postiz→us event path	HMAC/schema-validated, notification-only (ids), authoritative re-fetch	Medium
Cross-tenant data access	Broken isolation	FORCE-RLS third wall + Cerbos + scoped API (defense in depth)	Critical
Runaway metered spend	X fees, generative credits	tool_scope gate → budget stop-loss chain → fail-closed refusal	High
AI performs harmful action	Prompt injection via brand corpus/content	Drafts only; server-side authz + WS4 gate; untrusted-input stance	Critical

Control layers

KEY CUSTODY (THREE-WAY SPLIT)

App creds (OpenBao → Postiz env) · Postiz org keys (server-side alias) · network tokens (inside Postiz only, never copied). Disjoint blast radii by construction.

AGPL CONTAINMENT

No shared code/DB/process, REST-only, lint-enforced zero Postiz deps, fork-touchpoint budget with review. License boundary = security boundary.

RLS & TENANCY

FORCE-RLS, third-wall predicate, fail-closed empty-set semantics on every `social_*` table except the `client-data-free` `social_platform_apps`.

WRONG-ACCOUNT DEFENSE

FK chain
variant→account→publisher_org→(validated at composer time
AND re-validated at the
dispatch choke-point.

PUBLISH SAFETY

Single dispatch choke-point:
scope check → media/quota
re-validate → one-shot
approvalId + hash match →
schedulePost → stamp →
true-up. No auto-retry of
ambiguous failures.

CERBOS RESOURCES

Seven new resource policies
(engagement, account, post,
inbox, report, ledger,
platform_app) with derived-
roles reuse; UI mirrors in
`lib/rbac.ts`.

INBOUND SURFACES

Edge exposes only exact-path
OAuth callbacks/webhooks to
Postiz; Postiz→us webhooks
notification-only,
HMAC/secret-path validated.

MONEY SAFETY

Estimate → tool-scope →
ledger stop-loss → dispatch
→ true-up. X spend visible on
the approval card before the
human clicks.

PUBLISH SAFETY – THE NON-OBVIOUS ONE

The AI ingests brand corpus and past-post content, which is **untrusted input**. Defense against prompt injection is not in the prompt — it is that **every outbound-public tool structurally suspends to WS4 regardless of what the model decided**, and execution requires a one-shot approval bound to an exact payload hash. The model can propose; only an authorized, gated, hash-matched command executes — for humans and agents alike.

Audit

Every ledger row, every Cerbos decision (existing), every hub tool call (existing JSONL), every approval decision (existing), every publisher-adapter call (OTel + module log with org/network/op) — nothing new to invent.

§12 Rollout & Tickets

Risk-honest phase order, each shippable, each with a clear gate before the next. 27 tickets P0–P4, /army-ready.

P0

Foundation + containment

Migrations 0034+ (all \$04 tables + third-wall RLS), `social` ModuleContract skeleton, Cerbos policies, the Postiz containment spike + deploy, the `SocialPublisher` port + driver + connector-registry sync.

GATE → containment checklist verified (REST-only, zero deps, fork-touchpoint budget); tripwire assessment says thin-fork holds, else escalate to Mixpost fallback.

P1

Organic publish + calendar + composer (own accounts, \$0)

Account connect flow (own brand accounts first), composer backend (media-rule/quota validation, payload-hash), the WS4 publish gate (one-shot approval), publish dispatch + status reconcile, console shell (Calendar + Composer).

GATE → a scheduled post on an own account publishes for real; status/URL round-trips; e2e verified against a live account.

P2

Engagement inbox

Inbox sync (comments/mentions/DMs where supported), AI triage (sentiment/spike), reply flow reusing the P1 approval pattern, Inbox tab UI (triage, assignment, SLA timers).

GATE → full loop in-console: new comment → triaged → assigned → draft → approve → sent; SLA timer renders and breaches visibly.

P3

AI copy/assets + analytics/reports (+ X metering)

Brand-voice RAG + drafting services, Creative Image Studio seam, metrics pull + Analytics tab, X metering live + ledger stop-loss, monthly reports (AI narrative → approve → deliver).

GATE → scripted sequence reconciles ledger vs dispatches; monthly run produces an approvable, deliverable report.

P4

Agent-proposed drafts

Full MCP agent surface hardened for automation principals (OBO), `smm-agent-content-brief` flow, best-time-to-post classical-stats job.

GATE → an agent goal produces a week of draft posts + imagery within credits, landing entirely in the approvals inbox; an agent attempt to publish directly suspends.

Indicative sequencing

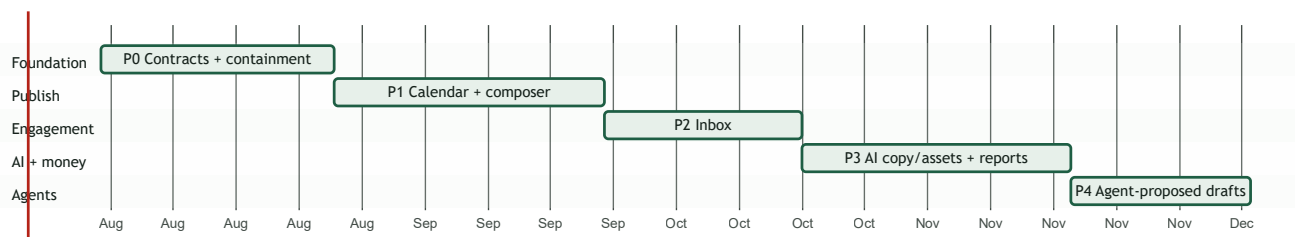


FIG. 12-1 – Indicative timeline; durations are planning estimates, not commitments. Sequencing (dependencies) is the fixed part.

Sample ticket decomposition (P0 — Foundation + containment)

ID	TICKET	TIER	MODEL	DONE WHEN (AC)
SMM-01 ⚡	Migrations 0034+: all \$04 tables + third-wall RLS + <code>social_platform_apps</code>	<code>senior-db</code>	<code>opus·medium</code>	RLS suite proves right-tenant→rows, cross-tenant→zero; <code>UNIQUE(postiz_org_id)</code> enforced
SMM-02 ⚡	<code>social</code> ModuleContract + controller skeleton + CRUD + tool-scope endpoints	<code>senior-be</code>	<code>default</code>	Module registers; scope toggles round-trip; disabled-module tenant gets 404s
SMM-03 ⚡	Cerbos policies ×7 + derived- roles + <code>lib/rbac.ts</code> mirror	<code>medior</code>	<code>default</code>	Parity tests incl. publish/reply/delete_published denials

ID	TICKET	TIER	MODEL	DONE WHEN (AC)
SMM-04 ⚡	Postiz containment spike + deploy; edge path-allowlist; DM-coverage assessed	senior-integrator	opus · medium · QA gate	UI unreachable from any routed surface; containment-audit note written; tripwire assessment says thin-fork holds
SMM-05 ⚡	SocialPublisher port + Postiz driver + org provisioning + registry sync	senior-be	opus · medium	Cross-client FK-chain validation refuses a mismatched account; zero postiz deps (lint asserted)
SMM-06	Config plumbing: key aliases, caps env, per-network feature flags	junior	default	Boots with and without Postiz reachable; clean degrade

Full decomposition – 27 tickets across P0–P4: senior-db 1 · senior-be 8 · senior-fe 2 · senior-integrator 2 · medior 11 · junior 3. Opus flags: 4 (SMM-01 med, SMM-04 med, SMM-05 med, SMM-09 high – the WS4 publish gate). Decision-gated, not mobilized: SMM-28 Mixpost fallback swap, SMM-29 video repurposing. Concurrency respects the 1–2 agent cap; SMM-09 (the publish spine) must run alone.

NON-CODE PARALLEL TRACK

Platform-app review submissions (OQ-1) run from day one as a parallel non-code workstream — they gate client-account connects, not the build. Registration in MODULES.md as social-media · 0.0.0 · PLANNED happens on approval of this doc; first merged ticket flips it to IN PROGRESS + CHANGELOG entry.

§13 Open Questions

Owner decisions — none block the build; several gate client-facing spend or connects.

#	QUESTION	BLOCKS	DEFAULT IF UNANSWERED
OQ-1	Platform-app review pipeline — which networks first, under which legal entity, who owns the weeks-long Meta/TikTok/LinkedIn review + business-verification work?	Client-account connects (not the build)	Submit Meta (IG/FB) + LinkedIn now; TikTok after; YouTube quota with first video client
OQ-2	Enable X publishing at pay-per-use (~\$0.015/post, reverify)? Metering is built regardless.	X network toggle	Ship with networks.x disabled in every scope until a client needs it

#	QUESTION	BLOCKS	DEFAULT IF UNANSWERED
0Q-3	Counsel sign-off on the AGPL containment memo (\$06 invariants + fork-touchpoint budget + source-offer mechanics) before client accounts connect	Client-facing P1+ (own-account dogfood proceeds)	Proceed on own accounts only; hold client connects for sign-off; Mixpost Pro is the priced fallback
0Q-4	DM coverage: if SMM-04's spike finds Postiz's DM surface thin on key networks, accept comments+mentions-only v1 inbox?	SMM-15 scope	Comments+mentions v1; DMs where Postiz supports them; revisit per network
0Q-5	Video asset storage: local <code>files</code> + Drive mirror, or Drive-first with reference-attach?	SMM-08 media handling	files + Drive mirror (existing pattern)
0Q-6	Video service line timing — when does repurposing get gated open, is generative video wanted at all?	SMM-29	Park until a client deliverable demands it

§14 Decision Log

LOCKED UPSTREAM (FOUNDATION - NOT RELITIGATED HERE)

v1 scope = organic content-studio + publisher (posting, engagement, copywriting, digital assets); paid ads / listening / influencer OUT (seams noted); publisher = Postiz (AGPL-3.0, free) run AGPL-CONTAINED, Mixpost Pro paid fallback; Chatwoot dropped (no second inbox; wa-chat-bot untouched); copy via ai-gateway-go + pgvector brand-voice RAG; digital assets via the Creative Image Studio + gateway image seam; per-network app-review/credential/rate-limit reality modeled first-class; generative image/video behind approvals + credits; human-in-the-loop mandatory, stricter than SEO.

New decisions made by this design (overturn only with cause)

#	DECISION	WHY
D-1	Postiz = execution engine only: drafts, approvals, judgment, and all domain state live in platform-nest; a post enters Postiz only after WS4 approval, via one adapter	Makes the AGPL boundary and the approval boundary the same line — both structurally uncrossable
D-2	Tenant mapping = one Postiz org per (tenant, client) in our <code>social_publisher_orgs</code> (UNIQUE org), org-scoped API keys server-side; tenancy never forked into Postiz	Containment (tenancy IP stays ours) + wrong-account-publish defence
D-3	Module key <code>social</code> , tables <code>social_*</code> , hr-style controller path + third-wall RLS, migrations 0034+	Newest-vertical (hr) conventions; same as SEO D-3

#	DECISION	WHY
D-4	No shared no-RLS cache in this module; sole non-RLS table is the client-data-free <code>social_platform_apps</code>	All social data is client-private; the SEO exemption's justification does not transfer
D-5	Three-way credential split: platform-app creds (OpenBao → Postiz env) · Postiz org keys (server-side alias) · network tokens (inside Postiz only, never copied)	Disjoint blast radii; our DB/vault never holds a social token
D-6	Every outbound-public action = WS4 <code>impact:'high'</code> + one-shot approvalId + payload-hash match, consumed transactionally at dispatch; edits invalidate approvals; no auto-retry of ambiguous failures	Inherits SEO D-6, tightened for public irreversibility; replay- and TOCTOU-proof
D-7	Single-mode execution (no manual/api dual mode); hand-posted content enters as <code>native_import</code> bookkeeping	Publishing is inherently API-shaped; a "manual twin" would just be unbooked work
D-8	Per-engagement <code>tool_scope</code> + budget carried over from SEO D-11 (networks, cadence, inbox SLA, AI toggles, caps); every flow and metered dispatch consults it	Fully custom per-client service shape; cost = Σ enabled features
D-9	X per-post fees + generative credits + attributable cloud-text in one <code>social_usage_ledger</code> with the SEO stop-loss chain	One metering discipline platform-wide; X never bleeds silently
D-10	Console = the reserved Publish craft group (Calendar · Composer · Inbox · Analytics) on the dept template, dept name SMM; toolkit registered only when all four routes exist	IA already ratified in the dept-console plan; honors the toolkit file's own rule
D-11	Ingress = exact-path allowlisted edge route to Postiz OAuth callbacks/webhooks only; Postiz→us webhooks are notification-only, authoritative state re-fetched	Smallest possible inbound door; forgery yields only a redundant authenticated fetch
D-12	Composer owns pre-publish validation (media rules, quotas, cost) — we never rely on Postiz or the network to reject	Don't let users queue what the API will reject; failures surface before approval, not after
D-13	Brand corpus lives in WS8 knowledge (tenant-ACL'd); <code>social_brand_profiles</code> holds config/pointers only; no module-local embedding columns	D9 ownership preserved — nothing here is operational-clustering-shaped

§15 Glossary

TERM	MEANING
AGPL-CONTAINED	Run at arm's length behind one REST adapter — no shared code, DB, or process — so the platform is never infected by the license

TERM	MEANING
License zone	A trust-zone variant defined by license boundary, not network location — Postiz runs on the private network but is architecturally isolated
Connector registry	<code>social_accounts</code> — per client × network state (status/quota/health), mirrored from Postiz; modeled like the IT device registry
Two-level content model	<code>social_posts</code> (master idea/brief) + <code>social_post_variants</code> (per-network content) — no universal post object
One-shot approvalId	A WS4 approval token bound to an exact payload hash, consumable exactly once at dispatch; any edit invalidates it
payload_hash	Hash of a variant's body/media/settings/schedule — the approval-match anchor; recomputed on every edit
tool_scope	Per-engagement jsonb config (networks, cadence, inbox SLA, AI toggles, caps) consulted by every flow and metered dispatch
Native import	A hand-posted item recorded after the fact as <code>published</code> with no approval trail — bookkeeping, not execution
Stop-loss chain	Fail-closed budget check: engagement cap → tenant cap → global cap, before any metered dispatch
Brand-voice RAG	Retrieval over a tenant-ACL'd WS8 knowledge corpus of past posts and guidelines, used to keep AI drafts on-brand
WS4 approvals	Existing ERP human-approval surface reused to gate every outbound-public social action
FORCE-RLS third wall	<code>tenant_id = ANY(app_current_tenants()) AND app_module_allowed('social')</code> on every tenant table

GAIADA Social Media — Engineering Blueprint v1.0

Status: Draft for review · 2026-07-23 · Social Media · Classification: Internal
Architecture decisions locked (§14). Feeds architect decomposition → /army. Not a commitment of dates; sequencing is fixed, durations are estimates.